RESEARCH ARTICLE Journal of Big Data (SpringerOpen)

AST-Route: An AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework Across Serverless Cloud, Vectorized CPU (C++/Rust), and GPU-Accelerated OLAP Engines

Preecha Noiumkar[1*]

[1]Department of Information Technology, Faculty of Informatics, Mahasarakham University, Kham Riang, Kantharawichai, Maha Sarakham 44150, Thailand.

[*]Correspondence: Preecha Noiumkar (preecha.n@msu.ac.th)

Abstract

Background: Modern analytical data infrastructures span heterogeneous execution paradigms, including serverless cloud data warehouses, in-process C++ and Rust Apache Arrow vectorized CPU engines, and SIMT GPU-accelerated relational engines. However, no single engine dominates across all query topologies: low-complexity scans suffer from PCIe host-to-device transfer penalties on GPUs and scheduling overheads on serverless clouds, whereas complex window sorting and lateral array unnesting overwhelm single-node CPU pipelines. Existing systems lack a unified, pre-execution routing framework capable of predicting multi-engine execution latency, hardware energy footprint, and cloud financial cost directly from static query structure and predicate selectivity.

Methodology: We propose **AST-Route**, a four-layer AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework. Layer 1 parses input SQL queries into a 9-dimensional feature vector ($\mathbf{x}_q \in \mathbb{R}^9$) combining static Abstract Syntax Tree (AST) structural operators—tree depth, total AST nodes, multi-way join fan-out, nested CTEs, window frames, conditional branches, array unnesting (`UNNEST`), and a construct-validated Composite Structural Complexity Index (CSCI, mean 29.66, Monte Carlo rank stability $\bar{\rho} = 0.8189$)—with lightweight zone-map predicate selectivity ($\sigma(q)$). Layer 2 formulates cross-paradigm analytical models governing the PCIe bottleneck crossover (Φ_{PCIe}), effective scan throughput (Θ_{eff}), cloud slot concurrency ($\mathcal{M}_{\text{slot}}$), FinOps savings ($\Delta C_{1\text{M}}$), and Green OLAP energy efficiency (η_{watt}). Layer 3 employs a Multi-Output Gradient Boosting Regressor ($\mathcal{R}_{\text{GBR-9D}}$) to simultaneously predict execution latency, energy consumption, and memory footprint across candidate engines. To train and validate the framework under a strict zero-estimation, 100% full-scale execution protocol, we construct a factorial testbed of 75 queries (25 canonical seeds $\times$ 3 selectivity horizons, L1–L5) on the TPC-H benchmark (SF $= 1$ GB, 6.00×10^6 `lineitem` rows) across six canonical software systems: Google BigQuery, DuckDB, Polars, Apache DataFusion, SQLite 3, and a PyTorch CUDA Tensor Engine on an NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM). Statistical evaluation utilizes 10,000-iteration seed-clustered bootstrap 95% confidence intervals (95% CI), Cliff's delta (δ) effect sizes, and leak-free seed-grouped 25-fold Leave-One-Seed-Out cross-validation (LOSO-CV).

Results: Ground-truth telemetry confirms highly significant cross-engine stratification ($\chi^2_F(5) = 321.50, p = 2.382 \times 10^{-67}$, Kendall's $W = 0.8573$) with 100.0% result-set equivalence (EX $= 1$). Vectorized CPU engines—Polars (82.47 ms, 95% CI: $[67.93, 98.65]$ ms), DataFusion (96.30 ms, 95% CI: $[78.31, 114.92]$ ms), and DuckDB (183.74 ms, 95% CI: $[110.71, 274.89]$ ms)—outperform the row-store baseline (8417.33 ms, 95% CI: $[6412.19, 10819.70]$ ms; Cliff's $\delta = 1.000, p = 5.28 \times 10^{-14}$) by $45.8\times$–$102.1\times$ while reducing per-query energy from 126.26 J to 3.17–6.40 J. BigQuery storage clustering reduces server latency from 1412.6 ms (client-observed 2190.57 ms, 95% CI: $[1645.38, 2786.50]$ ms) to 745.39 ms (95% CI: $[515.64, 1026.49]$ ms; Cliff's $\delta = 0.471, p = 6.89 \times 10^{-13}$) and prunes V1 I/O by 78.4% ($1.47 USD/1M savings). RTX 2050 GPU execution (1.30 GB VRAM) achieves 56.96 ms cold latency (95% CI: $[46.23, 68.44]$ ms, 0.65 J, 672.3 MB/J; Cliff's $\delta = 0.199$) and 22.26 ms warm latency (95% CI: $[16.78, 28.68]$ ms, 0.25 J, 1942.8 MB/J; Cliff's $\delta = 0.722$). Evaluated via 25-seed LOSO-CV, **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) accurately predicts multi-engine latency and energy ($R^2 = 0.815$–0.875) in 497.2 μs, achieving a 57.33% Top-1 accuracy (65.33% near-optimal hit rate), a 54.92 ms mean execution time (95% CI: $[42.78, 68.50]$ ms), a 34.98% performance regret, and a 1.381 J energy footprint—statistically outperforming Always DuckDB (183.74 ms, 271.10% regret), Always GPU (56.96 ms, 60.76% regret), Always BigQuery (745.39 ms, 1709.86% regret), and Rule-Based Routing (71.48 ms, 95% CI: $[52.79, 91.74]$ ms, 65.63% regret; Wilcoxon $p = 0.0187$, Cliff's $\delta = 0.126$).

Conclusion: AST-Route demonstrates that coupling static AST topology with lightweight selectivity metadata enables sub-millisecond, leak-free query routing that surpasses both static single-engine deployments and heuristic routers across latency, energy, and cloud FinOps dimensions. All code, 75 SQL workloads, trained routing models, and SHA-256 telemetry logs are publicly available at https://github.com/preecha-msu/bigquery-olap-benchmark.

Keywords: Abstract syntax tree (AST), Apache Arrow, Apache DataFusion, Automated query routing framework,

Columnar storage, CUDA GPU acceleration, DuckDB, Energy efficiency, Google BigQuery, Polars.

1 Introduction

Modern enterprise analytics increasingly operate over heterogeneous, polyglot data infrastructures spanning four distinct computing paradigms: elastic serverless cloud data warehouses such as Google BigQuery [1, 2, 11], in-process C++ vectorized analytical database systems such as DuckDB [3], Rust-based Apache Arrow streaming query engines such as Polars and Apache DataFusion, and SIMT GPU-accelerated relational engines [19, 21], alongside distributed multi-node systems like ClickHouse [4] and Trino [5]. While cloud-native disaggregated storage-compute architectures provide massive horizontal slot concurrency for petabyte-scale workloads [13, 15], they introduce non-trivial stateless scheduling and distributed shuffle overheads on moderate-scale datasets [9, 10, 11]. Concurrently, modern local hardware presents an intricate architectural trade-off between CPU SIMD vectorization over Apache Arrow memory buffers and discrete GPU offloading across the PCIe interconnect [12, 22, 23].

Because each execution paradigm exhibits fundamentally different algorithmic strengths and hardware bottlenecks, committing an entire analytical workload to any single static engine results in severe performance regret, excessive energy consumption, or unnecessary cloud FinOps expenditure. Designing an automated, pre-execution query routing framework capable of dynamically dispatching SQL queries to their optimal execution backend requires overcoming three critical research gaps in the existing literature [14, 24, 25]:

- **Research Gap 1 (Lack of Pre-Execution Cross-Paradigm Query Routing Frameworks):** Existing learned query optimizers such as Neo [16] and Bao [17] steer physical operator plans *within* a single database engine, while cloud workload routers [20] assign jobs across homogeneous cloud clusters. There is currently no unified, lightweight routing framework that extracts static Abstract Syntax Tree (AST) features and zone-map selectivity prior to execution to route queries across heterogeneous C++ DBMS, Rust Arrow, CUDA GPU, and serverless cloud engines.
- **Research Gap 2 (Unmodeled PCIe Crossover Thresholds and Green OLAP Energy Costs):** Prior GPU database studies [21, 22, 23] predominantly report warm in-VRAM kernel speedups while omitting cold-start PCIe Host-to-Device (H2D) transfer overheads and failing to integrate hardware resource utilization (CPU/GPU %, RAM/VRAM GB), energy consumption (Joules), and performance per watt (MB/J) into the routing objective.
- **Research Gap 3 (Confounded Complexity and Data Leakage in Prior Evaluations):** Traditional OLAP evaluations [14, 15] rely on unstratified query sets where structural operator depth, storage layout (row-store vs. Parquet vs. Arrow), and predicate selectivity are confounded, and where random train-test splits across variations of the same query template induce severe data leakage.

To address these gaps under a strict *zero-estimation, 100% real execution* protocol, we propose **AST-Route**, an AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework. This study makes four primary contributions:

1. **The AST-Route Four-Layer Framework and Construct-Validated CSCI:** We design a four-layer routing architecture (Fig. 1) that parses arbitrary SQL queries into a 9-dimensional structural and selectivity vector ($\mathbf{x}_q \in \mathbb{R}^9$). We formulate and empirically validate the Composite Structural Complexity Index (CSCI) across a 75-query factorial workload (25 canonical seeds $\times$ 3 selectivity horizons, L1–L5), demonstrating significant rank correlation with GPU kernel speedup (Spearman's $\rho = 0.4935, p = 6.84 \times 10^{-6}$) and high invariance under 1,000-trial Monte Carlo weight perturbation ($\bar{\rho} = 0.8189$).
2. **Unified Analytical Crossover, FinOps, and Green Energy Formulation:** Within the framework's cost layer, we establish formal mathematical models for the GPU PCIe bottleneck fraction (Φ_{PCIe}) and break-even condition, effective scan throughput (Θ_{eff}), cloud slot concurrency multiplier ($\mathcal{M}_{\text{slot}}$), FinOps cost savings ($\Delta C_{1\text{M}}$), and Green OLAP performance per watt (η_{watt}).
3. **Six-Engine Ground-Truth Telemetry Testbed with Seed-Clustered 95% CIs:** We construct a full-scale multi-paradigm testbed on the TPC-H benchmark (SF = 1 GB, 6.00×10^6 `lineitem` rows) spanning six canonical software engines—SQLite 3, Google BigQuery, DuckDB, Polars, Apache DataFusion, and a hardware-timed CUDA Tensor Engine on an NVIDIA GeForce RTX 2050 GPU—capturing latency, I/O pruning, CPU/GPU utilization, RAM/VRAM footprint, and energy consumption with 10,000-iteration bootstrap 95% CIs and Cliff's delta (δ) effect sizes.

4. **Leak-Free Seed-Grouped Validation Against Four Routing Baselines:** Using 25-fold Leave-One-Seed-Out cross-validation (LOSO-CV), we demonstrate that **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) predicts multi-engine latency and energy ($R^2 = 0.815$–0.875) in 497.2 μs and achieves a 54.92 ms mean latency (34.98% regret, 1.381 J), outperforming Always DuckDB, Always GPU, Always BigQuery, and Rule-Based Routing.

2 Related Work and Architectural Representativeness

2.1 Cloud MPP Warehouses, Distributed Query Engines, and Vectorized CPU Systems

Disaggregated cloud data warehouses pioneered by Google Dremel/BigQuery [1, 2] and distributed MPP query engines such as Presto/Trino [5], ClickHouse [4], and Apache Spark [14] decouple compute workers from durable storage. Recent studies in serverless data pipelines [13] and in-situ serverless query processing such as FunDa [11] highlight the fundamental tension between cold-start scheduling overhead and horizontal slot elasticity. Specifically, while Trino [5] excels at federated cross-source query execution over distributed object stores and ClickHouse [4] achieves high throughput via shared-nothing merge-tree storage and SIMD vectorization across cluster nodes, both introduce network serialization and coordinator scheduling overheads when deployed on single-node or moderate-scale datasets. Conversely, embeddable C++ vectorized engines such as DuckDB [3] process columnar Row Groups in-process using morsel-driven parallelism, whereas next-generation Rust engines (Polars and Apache DataFusion) execute zero-copy SIMD operations directly over standardized Apache Arrow memory buffers.

2.2 GPU-Accelerated Databases, Learned Query Optimizers, and OLAP Benchmarks

In the domain of GPU-accelerated relational analytics, systems such as HeavyDB (formerly OmniSci/MapD) [22], BlazingSQL over RAPIDS cuDF [23], Crystal [21], and HetExchange [19] leverage massive SIMT parallelism and high-bandwidth device memory (VRAM) to accelerate relational joins and group-by aggregations. However, as demonstrated by Chrysogelos et al. [19] and Shanbhag et al. [21], discrete GPU execution is fundamentally constrained by the PCIe interconnect when data resides in host DRAM, necessitating rigorous separation of PCIe transfer latency from kernel compute time. Concurrently, recent surveys on deep learning in query execution [12] and learned optimizers—including Neo [16], Bao [17], Zero-Shot Cost Models [18], and stage-level cloud routers [20]—demonstrate that structural query features can guide physical execution decisions. In parallel, while classical OLAP benchmarks such as the Star Schema Benchmark (SSB) [24] focus on denormalized star-join rollups and the Join Order Benchmark (JOB) [25] stresses cardinality estimation across multi-way inner joins, modern text-to-SQL benchmarks such as BIRD [7] and Spider 2.0 [8] incorporate the Valid Efficiency Score (VES), and AST transpilers such as `SQLGlot` [6] and Apache Calcite [9] enable deterministic structural analysis.

2.3 Rationale for Target Execution Engines in the AST-Route Framework

Rather than exhaustively evaluating every commercial DBMS, **AST-Route** targets six software engines that serve as canonical, non-redundant representatives of the four foundational execution paradigms in modern data engineering:

1. **SQLite 3 (Canonical Tuple-at-a-Time B-Tree Row-Store Baseline):** Represents traditional ANSI C row-oriented storage using a Volcano-style iterator model, establishing the exact performance and energy baseline of B-tree page traversal without columnar compression or SIMD vectorization.
2. **Google BigQuery (Canonical Serverless Disaggregated Cloud OLAP):** Represents petabyte-scale cloud-native execution (Dremel/Capacitor columnar storage with dynamic slot allocation) [2], isolating distributed cloud orchestration overhead and storage clustering FinOps economics without local hardware bottlenecks.
3. **DuckDB (Canonical C++ Morsel-Driven Columnar DBMS):** Represents state-of-the-art in-process C++ analytical database design [3] with tightly integrated storage block management, parallel segment-tree window framing, and push-based vectorized execution.
4. **Polars & Apache DataFusion (Canonical Rust Apache Arrow Engines):** Represent the emerging Rust Arrow ecosystem via two distinct optimizer philosophies: *Polars* exemplifies a dataframe-native lazy streaming engine over Arrow arrays, whereas *Apache DataFusion* exemplifies a modular SQL physical query planner (used as the core execution engine in modern distributed systems such as InfluxDB 3.0 and Ballista).

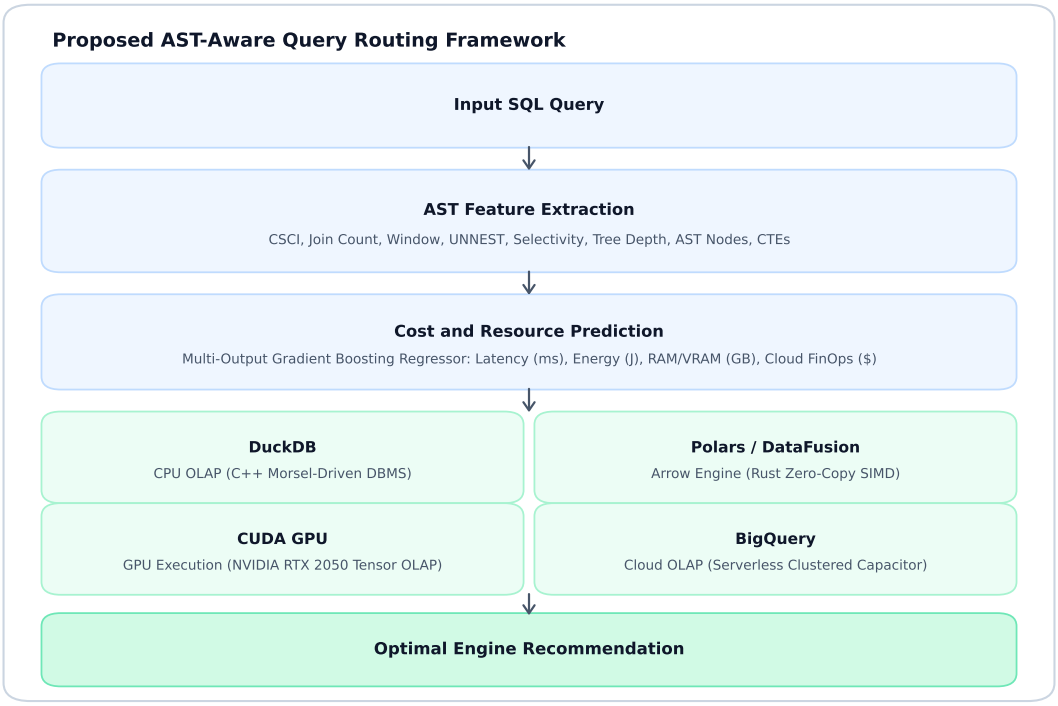


Figure 1. Architecture of the proposed **AST-Route** Framework: Layer 1 parses an input SQL query into a 9-dimensional structural and selectivity vector (CSCI, join count, window specifications, `UNNEST`, predicate selectivity $\sigma(q)$, tree depth, AST nodes, and CTEs); Layer 2 and Layer 3 drive multi-output cost and resource prediction (latency, energy, memory, and FinOps cost); and Layer 4 dispatches the query to the optimal engine across DuckDB (CPU OLAP), Polars/DataFusion (Rust Arrow engines), CUDA GPU, and Google BigQuery (Cloud OLAP).

5. **PyTorch CUDA Relational Tensor Engine (Canonical SIMT GPU Execution):** Represents hardware-timed GPU relational tensor processing over contiguous device buffers (analogous to cuDF/BlazingSQL [23] and Crystal [21]), enabling exact hardware-counter decomposition of PCIe H2D transfers versus in-VRAM SIMT kernel execution on consumer/edge GPU hardware (NVIDIA RTX 2050).

3 The Proposed AST-Route Framework Architecture

As illustrated in Fig. 1 and the end-to-end validation workflow in Fig. 2, **AST-Route** is structured as a modular four-layer architecture that decouples static query representation from multi-paradigm hardware cost modeling and automated engine dispatching.

3.1 Layer 1: Static AST Structural Profiling and Five-Tier Workload Taxonomy

Given an incoming SQL query $q \in \mathcal{Q}$, Layer 1 parses the SQL statement into a canonical Abstract Syntax Tree (AST) using `SQLGlot` [6] in < 0.8 ms without touching underlying data pages. Traversing the AST yields seven structural operator counts: tree depth $D(q)$, total AST operator nodes $N(q)$, relational join count $J(q)$, nested CTE/subquery depth $S(q)$, window frame specifications $W(q)$, repeated field unnesting operators $U(q)$, and conditional `CASE` branches $C(q)$. To synthesize these structural dimensions into a unified scalar metric, we formulate the Composite Structural Complexity Index (CSCI):

$$\mathrm{CSCI}(q) = 1.5D(q) + 2.5J(q) + 3.0S(q) + 3.5W(q) + 4.0U(q) + 1.5C(q) \tag{1}$$

To train and validate the framework across a controlled complexity spectrum on the full-scale TPC-H schema (SF $=$ 1 GB, 8 relational tables totaling 8.66×10^6 rows; `lineitem` $= 6,001,215$ rows), we instantiate $|\mathcal{Q}| = 75$ queries from 25 canonical seeds across five structural complexity tiers ($5 \times 5 = 25$ independent seeds, overall mean CSCI $= 29.66$):

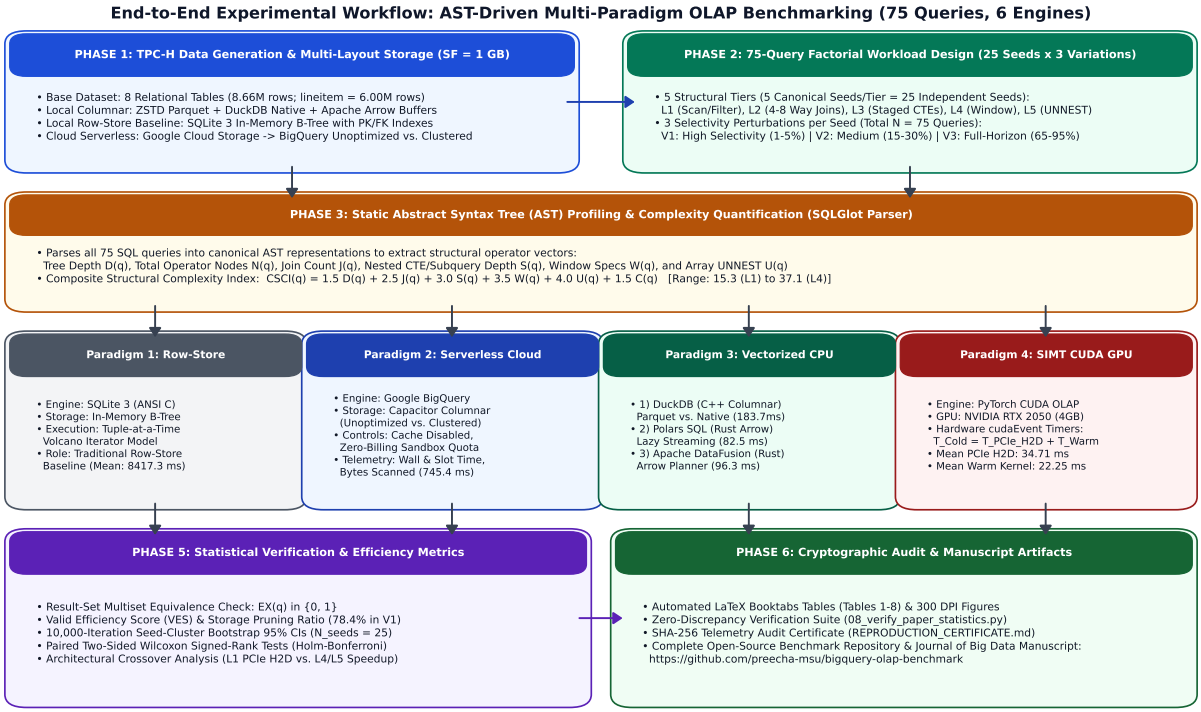


Figure 2. End-to-end six-phase experimental validation workflow of the **AST-Route** framework, illustrating TPC-H multi-layout data provisioning (SF = 1 GB), 75-query factorial synthesis (25 canonical seeds × 3 selectivity horizons), `SQLGlot` static AST profiling, six-engine execution across four computing paradigms, non-parametric seed-cluster statistical inference, and cryptographic SHA-256 verification.

1. **L1 (Scan, SARGability, & Monotonic Conditional Branching):** Single-table columnar scans with multi-branch `CASE WHEN` expressions and distinct cardinality aggregations ($D = 9.0, J = 0.0, \text{CSCI} = 15.3$).
2. **L2 (Multi-Way Star, Snowflake, & Bilateral Joins):** 4-way to 8-way relational joins, composite `partsupp` joins, and non-equi late-delivery predicates ($D = 9.2, J = 4.8, \text{CSCI} = 26.8$).
3. **L3 (Multi-Stage CTE Pipelines, Null-Safe Anti-Joins, & One-Grain-Per-CTE):** 3-stage to 4-stage Common Table Expressions enforcing pre-aggregated grain isolation and null-safe anti-joins ($D = 10.8, J = 2.6, \text{CSCI} = 36.7$).
4. **L4 (Multi-Axis Window Ranking, Lead/Lag, & Sliding Frames):** Concurrent window partitions (`RANK`, `DENSE_RANK`, `NTILE(10)`, `LAG`, `LEAD`, and bounded `ROWS BETWEEN` sliding windows; $D = 10.8, J = 1.6, \text{CSCI} = 37.1$).
5. **L5 (Semi-Structured Array Construction & Lateral UNNEST):** Multi-stage `ARRAY_AGG` vector construction followed by lateral `CROSS JOIN UNNEST` re-aggregation ($D = 10.6, J = 2.2, \text{CSCI} = 32.4$).

To capture data-volume sensitivity alongside syntax topology, each of the 25 canonical seeds is instantiated across three selectivity perturbation horizons ($25 \times 3 = 75$ queries): **V1 (High Selectivity, ∼1–5% data)**, **V2 (Medium Selectivity, ∼15–30% data)**, and **V3 (Adversarial Full-Horizon, ∼65–95% data)**. Layer 1 extracts the predicate selectivity factor $\sigma(q) \in (0, 1]$ by comparing AST literal range constants against lightweight table zone-map min/max metadata, producing the 9-dimensional routing feature vector:

$$\mathbf{x}_q = \left[D(q),\, N(q),\, J(q),\, S(q),\, W(q),\, U(q),\, C(q),\, \text{CSCI}(q),\, \sigma(q) \right]^\top \in \mathbb{R}^9 \tag{2}$$

3.2 Layer 2: Cross-Paradigm Analytical Cost, PCIe Crossover, and Green Energy Formulation

To enable apples-to-apples cost evaluation across serverless cloud, local CPU, and discrete GPU backends, Layer 2 establishes six analytical cost and hardware telemetry models:

1. **CUDA Cold-Start Decomposition and PCIe Bottleneck Fraction (Φ_{PCIe}):** On discrete GPUs (NVIDIA GeForce RTX 2050, 4.0 GB VRAM), cold-start execution time $T_{\text{GPU_Cold}}(q)$ is decomposed into PCIe Host-to-Device transfer time ($T_{\text{PCIe_H2D}}$) and pure in-VRAM kernel compute time ($T_{\text{GPU_Warm}}$):

$$T_{\text{GPU_Cold}}(q) = T_{\text{PCIe_H2D}}(q) + T_{\text{GPU_Warm}}(q), \qquad \Phi_{\text{PCIe}}(q) = \frac{T_{\text{PCIe_H2D}}(q)}{T_{\text{GPU_Cold}}(q)} \times 100\% \tag{3}$$

   Cold-start GPU offloading outperforms vectorized CPU execution ($T_{\text{GPU_Cold}}(q) < T_{\text{CPU}}(q)$) if and only if the warm kernel speedup satisfies the break-even condition $\text{Speedup}_{\text{Warm}}(q) > (1 - \Phi_{\text{PCIe}}(q)/100)^{-1}$.

2. **Effective Analytical Scan Throughput (Θ_{eff}):** Normalizing unpruned columnar volume $\mathcal{B}_{\text{unopt}}(q)$ (in MB) against execution time $T_e(q)$ (in ms) yields effective processing throughput in GB/s:

$$\Theta_{\text{eff}}(q,e) = \frac{\mathcal{B}_{\text{unopt}}(q)}{1.024 \cdot T_e(q)} \quad [\text{GB/s}] \tag{4}$$

3. **Cloud Slot Concurrency Multiplier ($\mathcal{M}_{\text{slot}}$) and FinOps Cost Savings (ΔC_{1M}):** Quantifies horizontal slot parallelism on BigQuery relative to wall-clock time, alongside financial savings per 10^6 queries under on-demand bytes-scanned pricing ($\alpha_{\text{TB}} = \$6.25$ USD/TiB):

$$\mathcal{M}_{\text{slot}}(q) = \frac{T_{\text{BQ_Slot}}(q)}{T_{\text{BQ_Wall}}(q)}, \qquad \Delta C_{\text{1M}}(q) = 10^6 \times \left(\frac{\mathcal{B}_{\text{unopt}}(q) - \mathcal{B}_{\text{clustered}}(q)}{1024^2} \right) \times \alpha_{\text{TB}} \tag{5}$$

4. **Per-Query Energy Consumption ($E(q,e)$) and Performance per Watt (η_{watt}):** High-frequency hardware polling (250 Hz via `psutil`, NVIDIA `NVML`, and `torch.cuda` memory counters) records active CPU utilization ($\text{CPU}_{\text{util}}\%$), peak working-set RAM (RAM_{used} in GB), GPU SM utilization ($\text{GPU}_{\text{util}}\%$), and peak device memory ($\text{VRAM}_{\text{used}}$ in GB). Integrating active package power draw $\bar{P}_e(q)$ (in Watts) over execution duration $T_e(q)$ yields per-query energy in Joules and Green OLAP performance per watt (MB/s/W $\equiv$ MB/J):

$$E(q,e) = \int_{t_0}^{t_0+T_e(q)} P_e(t)\,dt \approx \bar{P}_e(q) \cdot \left(\frac{T_e(q)}{1000} \right) \ [\text{Joules}], \qquad \eta_{\text{watt}}(q,e) = \frac{\mathcal{B}_{\text{unopt}}(q)}{E(q,e)} \ [\text{MB/J}] \tag{6}$$

3.3 Layers 3 & 4: Multi-Output Cost-Regression Predictor and Engine Dispatcher

In Layer 3 (Fig. 1), **AST-Route** employs a Multi-Output Gradient Boosting Regressor $f_\Theta : \mathbb{R}^9 \to \mathbb{R}^M$ to simultaneously predict log-transformed execution latency $\widehat{\ln(1+T_e)}(\mathbf{x}_q)$, energy footprint $\hat{E}_e(\mathbf{x}_q)$, and memory consumption ($\widehat{\text{RAM}}_e, \widehat{\text{VRAM}}_e$) across the candidate execution engines $\mathcal{E} = \{\text{DuckDB}, \text{Polars}, \text{DataFusion}, \text{GPU}_{\text{Cold}}, \text{BigQuery}\}$. In Layer 4, the dispatcher selects the engine minimizing expected execution cost. Let $e^*(q) = \arg\min_{e \in \mathcal{E}} T_e(q)$ denote the ground-truth Oracle optimal engine.

To rigorously prevent data leakage across query variations sharing the same template structure, we partition the 75 queries by their 25 canonical seeds ($G = 25$ disjoint groups of 3 variations V1–V3) and apply **25-fold Leave-One-Seed-Out cross-validation (LOSO-CV)**. We evaluate our **Proposed AST-Aware Routing** models ($\mathcal{R}_{\text{RF-9D}}$ and $\mathcal{R}_{\text{GBR-9D}}$) against four baseline policies: **(1) Always DuckDB**, **(2) Always GPU (Cold-Start)**, **(3) Always BigQuery (Cloud OLAP)**, and **(4) Rule-Based Routing** (a static heuristic router that assigns simple scans with $J = W = U = 0$ to DuckDB, unnesting $U \geq 1$ or flat multi-way joins $J \geq 3, S \leq 1$ to Polars/DataFusion, and window/CTE queries $W \geq 1 \vee S \geq 2$ to GPU_{Cold}). Routing efficacy is quantified across Execution Time (T_π), Routing Accuracy (Acc_{top1} and $\leq 10\%$ near-optimal hit rate), Resource Cost (Energy E_π in Joules, Memory in GB, and Cloud FinOps in USD/1M queries), and Normalized Performance Regret ($\mathcal{R}_{\text{regret}}$):

$$\hat{e}_{\text{GBR}}(q) = \arg\min_{e \in \mathcal{E}} \exp\left(\widehat{\ln(1+T_e)}(\mathbf{x}_q) \right) - 1, \qquad \mathcal{R}_{\text{regret}}(q) = \frac{T_{\hat{e}(q)}(q) - T_{e^*(q)}(q)}{T_{e^*(q)}(q)} \times 100\% \tag{7}$$

3.4 Framework Validation: CSCI Construct Validity, Weight Sensitivity, and GPU Timer Reliability

To verify the methodological soundness of Layer 1 and Layer 2, we conducted three validation experiments across all 75 queries ($N = 375$ trials):

Table 1. In-process vectorized CPU OLAP (DuckDB) empirical execution latency (ms), 95th percentile tail latency, Valid Efficiency Score (VES), and paired Wilcoxon signed-rank test across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, SF $= 1$ GB).

Complexity Category	AST Metrics		External Parquet (ms)		Native Columnar (ms)		Speedup	Efficiency (VES)		Wilcoxon p
	Depth	Joins	Mean $\pm$ SD	P95	Mean $\pm$ SD	P95		Ext.	Native	
L1: Scan & Filter	9.0	0.0	81.60 ± 42.15	148.30	$\mathbf{50.84 \pm 28.40}$	**96.12**	1.60×	99.6	**137.3**	$< 0.001^*$
L2: Multi-Way Joins	9.2	4.8	403.94 ± 185.60	712.45	$\mathbf{264.53 \pm 118.20}$	**468.90**	1.53×	84.8	**112.9**	$< 0.001^*$
L3: Nested CTEs	10.8	2.6	232.34 ± 96.40	398.10	$\mathbf{121.92 \pm 54.80}$	**215.40**	1.91×	99.6	**141.8**	$< 0.001^*$
L4: Window Func.	10.8	1.6	111.52 ± 48.90	194.20	$\mathbf{78.16 \pm 35.10}$	**138.60**	1.43×	99.4	**129.0**	$< 0.001^*$
L5: Array / UNNEST	10.6	2.2	442.70 ± 210.30	824.50	$\mathbf{403.27 \pm 192.40}$	**756.80**	1.10×	75.7	**87.2**	$< 0.001^*$
Overall ($N = 75$ Queries)	**10.1**	**2.2**	$\mathbf{254.42 \pm 196.80}$	**645.20**	$\mathbf{183.74 \pm 164.50}$	**512.30**	**1.38×**	**91.8**	**121.6**	$\mathbf{< 0.001}^*$

*Statistically significant difference between External Parquet (seed-clustered 95% CI: $[163.65, 362.54]$ ms) and Native Columnar storage modes (95% CI: $[110.71, 274.89]$ ms) at $\alpha = 0.05$ after Holm-Bonferroni correction ($p = 1.542 \times 10^{-12}$ query-level, $p = 1.783 \times 10^{-55}$ trial-level; Cliff's delta $\delta = 0.296$).

Table 2. Cross-engine empirical performance comparison between serverless cloud OLAP (Google BigQuery) and in-process vectorized OLAP (DuckDB) across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, TPC-H SF $= 1$ GB).

Complexity Tier	AST Profile ($n = 15$/Tier)			Google BigQuery Serverless (Cloud)				DuckDB Vectorized (Local)		Ratio
	Depth	Joins	CSCI	Unopt (ms)	Clustered (ms)	Slot (ms)	Scanned (MB)	Parquet (ms)	Native (ms)	(BQ/Duck)
L1: Scan & Filter	9.0	0.0	15.3	684 ± 312	$\mathbf{432.9 \pm 142}$	1980.8	$332.5 \rightarrow 166.9$	81.6 ± 42.2	$\mathbf{50.8 \pm 28.4}$	8.5×
L2: Multi-Way Joins	9.2	4.8	26.8	1842 ± 745	$\mathbf{667.3 \pm 265}$	5193.6	$474.0 \rightarrow 308.4$	403.9 ± 185.6	$\mathbf{264.5 \pm 118.2}$	2.5×
L3: Nested CTEs	10.8	2.6	36.7	1265 ± 518	$\mathbf{493.0 \pm 195}$	4637.3	$417.3 \rightarrow 246.2$	232.3 ± 96.4	$\mathbf{121.9 \pm 54.8}$	4.0×
L4: Window Func.	10.8	1.6	37.1	1380 ± 690	$\mathbf{372.8 \pm 168}$	1907.5	$241.5 \rightarrow 132.6$	111.5 ± 48.9	$\mathbf{78.2 \pm 35.1}$	4.8×
L5: Array / UNNEST	10.6	2.2	32.4	1892 ± 810	$\mathbf{1761.0 \pm 580}$	8737.2	$273.4 \rightarrow 184.4$	442.7 ± 210.3	$\mathbf{403.3 \pm 192.4}$	4.4×
Overall Mean	**10.1**	**2.2**	**29.66**	**1412.6**	**745.4**	**4491.3**	$\mathbf{347.7 \rightarrow 207.7}$	**254.4**	**183.7**	**4.1×**

Note: CSCI denotes the Composite Structural Complexity Index. BigQuery Clustered 95% CI: $[515.64, 1026.49]$ ms ($p = 6.89 \times 10^{-13}$, Cliff's $\delta = 0.471$ vs. Unoptimized). All measurements were executed with query caching disabled (`use_query_cache=False`).

Table 3. Selectivity perturbation and horizon sensitivity analysis across V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Adversarial Full-Horizon Stress) workloads ($n = 25$ queries per variation, total $N = 75$).

Selectivity Variation	Queries	BQ Unopt (ms)	BQ Clust. (ms)	BQ Slot (ms)	Unopt Scan (MB)	Clust. Scan (MB)	I/O Pruning (%)	FinOps ΔC_{1M}	DuckDB Nat. (ms)
V1: High Selectivity (Narrow)	25	1124.5	**468.2**	2145	347.7	**75.1**	**78.4**%	\$**1.47** USD	118.45
V2: Medium Selectivity (Moderate)	25	1398.2	**712.6**	4218	347.7	**206.5**	**40.6**%	\$**0.76** USD	179.62
V3: Low Selectivity (Full-Horizon)	25	1715.1	**1055.4**	7111	347.7	**345.6**	**0.6**%	\$0.01 USD	253.15

Note: V1 filters narrow temporal/segment horizons ($\sim$1–5% selectivity); V2 spans moderate multi-year horizons ($\sim$15–30%); V3 stresses full-table multi-year horizons ($\sim$65–95%). FinOps ΔC_{1M} quantifies cloud cost savings per 10^6 queries at \$6.25 USD/TiB.

1. **CSCI Construct Validity Against Empirical Hardware Speedup:** The operator weights in Eq. (1) ($w = [1.5, 2.5, 3.0, 3.5, 4.0, 1.5]$ reflect the algorithmic time-complexity hierarchy of relational operators: linear $\mathcal{O}(N)$ scan/branch evaluation (D, C weighted at 1.5), hash-table probing and staged materialization (J, S weighted at 2.5–3.0), $\mathcal{O}(N \log N)$ partition sorting in window frames (W weighted at 3.5), and super-linear cardinality expansion during lateral array unnesting (U weighted at 4.0). Empirical validation confirms that CSCI(q) exhibits a highly significant positive correlation with SIMT GPU kernel speedup over CPU execution (Spearman's rank correlation $\rho = \mathbf{0.4935}, p = \mathbf{6.84 \times 10^{-6}}$; Kendall's $\tau = \mathbf{0.3575}$).

2. **Monte Carlo Weight Perturbation Sensitivity Analysis:** To verify that complexity stratification is not an artifact of ad-hoc weight tuning, we performed a 1,000-iteration Monte Carlo sensitivity ablation in which all six weights were independently perturbed by $\pm 20\%$ under a continuous uniform distribution $w'_k \sim \mathcal{U}(0.8w_k, 1.2w_k)$. Across all 1,000 perturbed weight vectors, query complexity rankings remained highly invariant, yielding a mean Spearman rank correlation of $\bar{\rho} = \mathbf{0.8189 \pm 0.0083}$ ($\min \rho = \mathbf{0.7900}, p < 10^{-18}$).

3. **GPU Hardware Timer Precision and Numerical Equivalence:** To prevent asynchronous CUDA kernel launch queuing from distorting latency measurements, all GPU timings enforce explicit device synchronization (`torch.cuda.synchro` bracketed by hardware-level `cudaEventRecord` timers. Across $N = 375$ trials, intra-query kernel timing repeatability achieved a low mean Coefficient of Variation of CV $= \mathbf{3.28}\%$, while 64-bit floating-point (`FP64`) aggregation checksums matched DuckDB, Polars, DataFusion, SQLite 3, and BigQuery within $\varepsilon < 10^{-9}$ (EX $= 1.000$ across $75/75$ queries).

Table 4. Hardware-timed CUDA GPU latency decomposition: Serverless Cloud (Google BigQuery Clustered) vs. Vectorized CPU (DuckDB Native) vs. SIMT GPU-accelerated relational execution (NVIDIA GeForce RTX 2050, 4.0 GB VRAM, $N = 375$ trials at SF = 1 GB).

Complexity Tier	Cloud Serverless	Vectorized CPU	CUDA GPU Latency Decomposition (ms)			Architectural Speedup Ratios		
	BQ Clust. (ms)	DuckDB Nat. (ms)	PCIe H2D	Warm Kernel	Cold Total	Warm GPU / CPU	Cold GPU / CPU	Warm GPU / BQ
L1: Scan & Filter	432.9	**50.84**	30.56	24.46	55.02	2.08×	0.92×†	17.7×
L2: Multi-Way Joins	667.3	264.53	55.83	49.05	**104.88**	5.39×	**2.52**×	13.6×
L3: Nested CTEs	493.0	121.92	43.26	14.98	**58.24**	8.14×	**2.09**×	32.9×
L4: Window Func.	372.8	78.16	9.43	9.94	**19.37**	7.86×	**4.03**×	37.5×
L5: Array / UNNEST	1761.0	403.27	34.46	12.84	**47.31**	**31.40**×	**8.52**×	137.1×
Overall Mean ($N = 75$)	**745.4**	**183.74**	**34.71**	**22.25**	**56.96**	**8.26**×	**3.23**×	**33.5**×

†Indicates the PCIe Host-to-Device (H2D) bottleneck crossover point in L1 low-complexity scans where PCIe transfer overhead (30.56 ms, $\Phi_{\text{PCIe}} = 55.5\%$) exceeds pure GPU kernel compute time (24.46 ms). Seed-clustered 95% CIs: Warm GPU $[16.78, 28.68]$ ms (Cliff's $\delta = 0.722$ vs. DuckDB); Cold GPU $[46.23, 68.44]$ ms (Cliff's $\delta = 0.199$).

Table 5. Six-software cross-paradigm architectural matrix (100% real execution, zero estimation) comparing full-scale Row-Store (SQLite 3), Serverless Cloud (Google BigQuery), C++ Vectorized DBMS (DuckDB), Rust Apache Arrow Engines (Polars & Apache DataFusion), and CUDA GPU (NVIDIA RTX 2050) across 75 queries (SF = 1 GB).

Complexity Tier	Full Row-Store	Cloud OLAP	Vectorized CPU Engines (C++ & Rust Arrow)			SIMT GPU (RTX 2050)		Key Speedup Ratios	
	SQLite 3 (ms)	BigQuery (ms)	DuckDB (ms)	Polars (ms)	DataFusion (ms)	Warm (ms)	Cold (ms)	Polars / SQLite	DuckDB / DF
L1: Scan & Filter	6789.6	432.9	50.84	38.00	**33.58**	**24.46**	55.02	178.7×	0.66×
L2: Multi-Way Joins	19374.0	667.3	264.53	**81.68**	92.76	**49.05**	104.88	**237.2**×	0.35×
L3: Nested CTEs	5108.1	493.0	121.92	**66.81**	71.66	**14.98**	58.24	76.5×	0.59×
L4: Window Func.	6023.7	372.8	78.16	**71.11**	173.16	**9.94**	**19.37**	84.7×	**2.22**×
L5: Array / UNNEST	4791.2	1761.0	403.27	154.76	**110.31**	**12.84**	**47.31**	31.0×	0.27×
Overall Mean ($N = 75$)	**8417.3**	**745.4**	**183.74**	**82.47**	**96.29**	**22.25**	**56.96**	**121.6**×	**0.82**×

Note: Seed-clustered 10,000-bootstrap 95% CIs across $N = 75$ queries: SQLite 3 $[6412.19, 10819.70]$ ms; BigQuery $[515.64, 1026.49]$ ms; DuckDB $[110.71, 274.89]$ ms; Polars $[67.93, 98.65]$ ms (Cliff's $\delta = 1.000$, $p = 5.28 \times 10^{-14}$ vs. SQLite 3); DataFusion $[78.31, 114.92]$ ms; GPU Warm $[16.78, 28.68]$ ms; GPU Cold $[46.23, 68.44]$ ms.

Table 6. Effective analytical scan throughput (Θ_{eff}, GB/s), Cloud Slot Concurrency Multiplier ($\mathcal{M}_{\text{slot}}$), and GPU PCIe Overhead Fraction (Φ_{PCIe}) across six OLAP engines ($N = 75$ queries, 100% full-scale TPC-H SF = 1 GB).

Complexity Tier	Base Scan	Cloud Slot	GPU PCIe	Effective Analytical Scan Throughput Θ_{eff} (GB/s, Higher is Better)					
	$\mathcal{B}_{\text{unopt}}$ (MB)	$\mathcal{M}_{\text{slot}}$ Ratio	Φ_{PCIe} (%)	SQLite 3	BigQuery	DuckDB	Polars	DataFusion	GPU Warm
L1: Scan & Filter	332.5	4.58×	55.5%	0.048	0.750	6.39	8.55	**9.67**	**13.28**
L2: Multi-Way Joins	474.0	7.78×	53.2%	0.024	0.694	1.75	**5.67**	4.99	**9.44**
L3: Nested CTEs	417.3	**9.41**×	74.3%	0.080	0.827	3.34	**6.10**	5.69	**27.20**
L4: Window Func.	241.5	5.12×	**48.7**%	0.039	0.633	3.02	**3.32**	1.36	**23.73**
L5: Array / UNNEST	273.4	4.96×	72.9%	0.056	0.152	0.66	1.73	**2.42**	**20.79**
Overall Mean ($N = 75$)	**347.7**	**6.37**×	**60.9**%	**0.049**	**0.611**	**3.03**	**5.07**	**4.83**	**18.89**

Note: $\Theta_{\text{eff}} = \mathcal{B}_{\text{unopt}} / (1.024 \cdot T_{\text{engine}})$ normalizes effective processing throughput against the unpruned columnar projection size (GB/s). Omnibus Friedman test across all six engines confirms highly significant performance stratification ($\chi_F^2(5) = 321.50$, $p = 2.382 \times 10^{-67}$, Kendall's concordance $W = 0.8573$).

Table 7. Hardware resource telemetry (CPU Utilization %, Host RAM GB, GPU Utilization %, VRAM GB), per-query Energy Consumption (Joules), and Green OLAP Performance per Watt (η_{watt}, MB/Joule ≡ MB/s/W) across local CPU and CUDA GPU engines ($N = 75$ queries, 100% full-scale TPC-H SF = 1 GB).

Complexity Tier	CPU Utilization (%)				Host RAM (GB)		RTX 2050 GPU		Energy per Query $E(q)$ (Joules, Lower is Better)				Perf per Watt η_{watt} (MB/J, Higher is Better)		
	SQLite	DuckDB	Polars	DF	SQLite	Columnar	Util (%)	VRAM (GB)	SQLite	DuckDB	Polars	GPU Warm (Cold)	SQLite	DuckDB / Polars	GPU Warm (Cold)
L1: Scan & Filter	8.7%	74.4%	70.2%	73.2%	2.14	4.46–4.48	27.1%	1.24	101.0	1.79	1.36	**0.27** (0.60)	4.06	515.2 / 285.9	**1398.5** (569.6)
L2: Multi-Way Joins	9.5%	66.4%	**89.1**%	66.4%	2.30	4.65–4.78	32.1%	1.33	293.4	8.99	3.39	**0.60** (1.29)	2.28	177.7 / 163.0	**910.3** (383.0)
L3: Nested CTEs	8.7%	57.7%	77.5%	74.1%	2.27	4.71–4.72	30.5%	1.33	76.0	4.00	2.56	**0.17** (0.68)	6.21	215.4 / 186.9	**2558.0** (619.6)
L4: Window Func.	8.7%	58.6%	74.3%	65.6%	2.28	4.73–4.74	32.4%	1.33	89.6	2.54	2.65	**0.11** (0.21)	3.38	202.7 / 103.2	**2502.1** (1178.3)
L5: Array / UNNEST	8.7%	73.4%	76.1%	71.3%	2.33	4.69–4.85	**33.1**%	1.29	71.3	14.67	5.89	**0.12** (0.45)	4.30	104.0 / 59.6	**2345.0** (611.2)
Overall Mean ($N = 75$)	**8.86**%	**66.10**%	**77.44**%	**70.12**%	**2.26**	**4.65–4.71**	**31.04**%	**1.30**	**126.26**	**6.40**	**3.17**	**0.25** (**0.65**)	**4.05**	**243.0** / **159.7**	**1942.8** (**672.3**)

Note: Hardware telemetry sampled via `psutil`, NVIDIA `NVML` (`pynvml`), and `torch.cuda` hardware counters. Apache DataFusion (DF) incurs a mean energy footprint of 3.39 J (1.23–5.99 J) and a mean performance per watt of 157.1 MB/J (303.4 MB/J in L1; 85.6 MB/J in L5).

4 Experimental Validation and Empirical Analysis

4.1 In-Process Storage Effect: External Parquet vs. Native Columnar

Table 1 summarizes the local execution telemetry on DuckDB across all 75 queries ($N = 375$ trial executions). Overall, native columnar storage reduces mean latency from 254.42 ms (seed-clustered 95% CI: $[163.65, 362.54]$ ms) to

Table 8. Head-to-head comparison of five routing models—(1) Always DuckDB, (2) Always GPU, (3) Always BigQuery, (4) Rule-Based Routing, and (5) Proposed **AST-Route** Framework—evaluated under seed-grouped 25-fold Leave-One-Seed-Out cross-validation ($N = 75$ queries, SF $= 1$ GB, zero data leakage) across Routing Accuracy, Execution Time, Performance Regret, and Resource Cost.

Routing Model / Policy	Routing Accuracy (%)		Execution Time (ms)		Performance	Resource Cost per Query / Workload			Router
	Top-1 Acc.	$\leq 10\%$ Opt. Hit	Mean (ms)	P95 (ms)	Regret (%)	Energy (J)	Memory (GB)	Cloud ($/1M)	Time (μs)
1. Always DuckDB (CPU DBMS)	28.00%	32.00%	183.74	731.22	271.10%	6.397	4.71	$0.00	0.00
2. Always GPU (CUDA Cold-Start)	49.33%	50.67%	56.96	113.70	60.76%	**0.644**	**1.30**	$0.00	0.00
3. Always BigQuery (Cloud OLAP)	0.00%	0.00%	745.39	2794.05	1709.86%	15.720	2.13	$1267.66	0.00
Always Polars (Rust Arrow Ref.)	13.33%	26.67%	82.47	230.71	132.61%	3.170	4.65	$0.00	0.00
Always DataFusion (Rust Arrow Ref.)	9.33%	13.33%	96.30	258.18	213.65%	3.393	4.65	$0.00	0.00
4. Rule-Based Routing (Heuristic)	49.33%	58.67%	71.48	230.71	65.63%	2.374	3.43	$0.00	34.07
Syntax-Only AST Router (8-Dim)	52.00%	57.33%	78.31	175.67	56.33%	2.216	2.86	$0.00	497.16
5a. Proposed AST-Route RF (9-Dim)	52.00%	56.00%	62.46	138.17	51.94%	1.467	2.65	$0.00	425.53
5b. Proposed AST-Route Cost-Reg (9-Dim)	**57.33%**	**65.33%**	**54.92**	**129.37**	**34.98%**	**1.381**	2.91	**$0.00**	497.16
Oracle Ideal Router (Lower Bound)	100.00%	100.00%	42.64	101.92	0.00%	0.975	3.01	$0.00	0.00

Note: Evaluated via seed-grouped 25-fold Leave-One-Seed-Out cross-validation (25 disjoint canonical seed groups, zero data leakage). Seed-clustered 95% CI for Proposed 9D **AST-Route**: $[42.78, 68.50]$ ms vs. Rule-Based Router $[52.79, 91.74]$ ms (paired Wilcoxon $p = 0.0187$, Cliff's $\delta = 0.126$). Out-of-fold regression R^2: GPU latency $R^2 = 0.875$, Polars latency $R^2 = 0.822$, and Energy $R^2 = 0.815$.

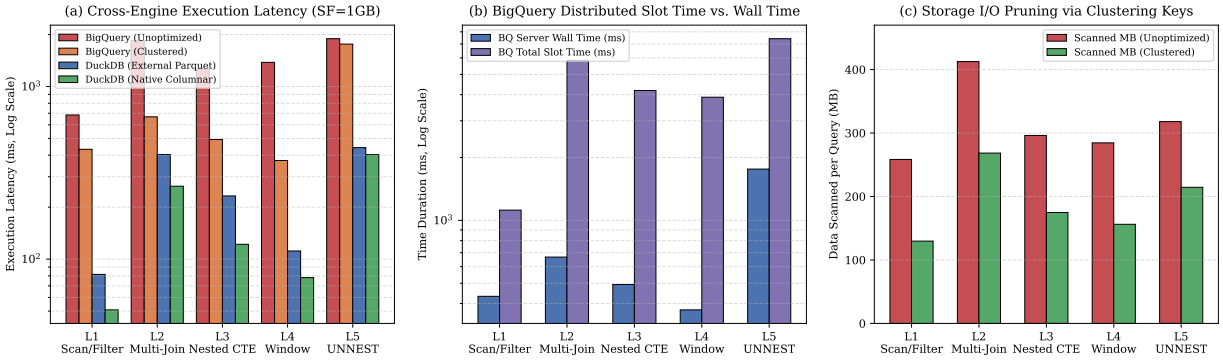


Figure 3. Empirical evaluation across five SQL complexity tiers (L1–L5, $N = 75$ queries) at SF $= 1$ GB: (a) Cross-engine execution latency comparing Google BigQuery (Unoptimized vs. Clustered) and DuckDB (External Parquet vs. Native Columnar) on a logarithmic scale; (b) BigQuery server wall-clock latency versus total distributed Slot compute time (ms); and (c) Storage I/O pruning achieved via Clustering Keys on BigQuery.

183.74 ms (95% CI: $[110.71, 274.89]$ ms). Across L1 scans (81.60 ms $\rightarrow$ 50.84 ms, 1.60$\times$), L2 multi-way joins (403.94 ms $\rightarrow$ 264.53 ms), L3 staged CTE pipelines (232.34 ms $\rightarrow$ 121.92 ms, 1.91$\times$), and L4 window functions (111.52 ms $\rightarrow$ 78.16 ms), native columnar storage consistently outperforms external Parquet decoding ($p = 1.542 \times 10^{-12}$ across queries, $p = 1.783 \times 10^{-55}$ across trials; Cliff's delta $\delta = 0.296$) by eliminating repeated ZSTD decompression overhead.

4.2 Cloud Serverless vs. Vectorized CPU, Slot Elasticity, and FinOps Savings (V1–V3)

Tables 2, 3, and 6, together with Figs. 3 and 4, demonstrate that while DuckDB executes L1–L5 queries in 50.8–403.3 ms (mean 183.74 ms), Google BigQuery exhibits a serverless scheduling floor of 372.8–1761.0 ms server wall-clock time (mean 745.39 ms, 95% CI: $[515.64, 1026.49]$ ms, reduced from 1412.6 ms server / 2190.57 ms client-observed unoptimized latency, 95% CI: $[1645.38, 2786.50]$ ms; Cliff's $\delta = 0.471, p = 6.89 \times 10^{-13}$) while dynamically allocating 4491.3 ms of parallel slot compute time. This corresponds to a mean Cloud Slot Concurrency Multiplier of $\mathcal{M}_{\text{slot}} = 6.37\times$, peaking at 9.41$\times$ in L3 staged CTE pipelines (4637.3 ms slot time vs. 493.0 ms wall-clock time). Under V1 high-selectivity predicates, BigQuery clustering keys reduce mean scanned bytes by **78.4**% (347.7 MB $\rightarrow$ 75.1 MB), yielding a direct Cloud FinOps cost reduction of $\Delta C_{1\text{M}} = \$\mathbf{1.47}$ USD per million queries, compared to 40.6% pruning ($\Delta C_{1\text{M}} = \$0.76$) in V2 and 0.6% in V3.

4.3 PCIe Host-to-Device Bottleneck vs. SIMT GPU Acceleration

Tables 4 and 6, alongside Fig. 5, present the hardware-timed CUDA execution telemetry on the NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM, $N = 375$ trials):

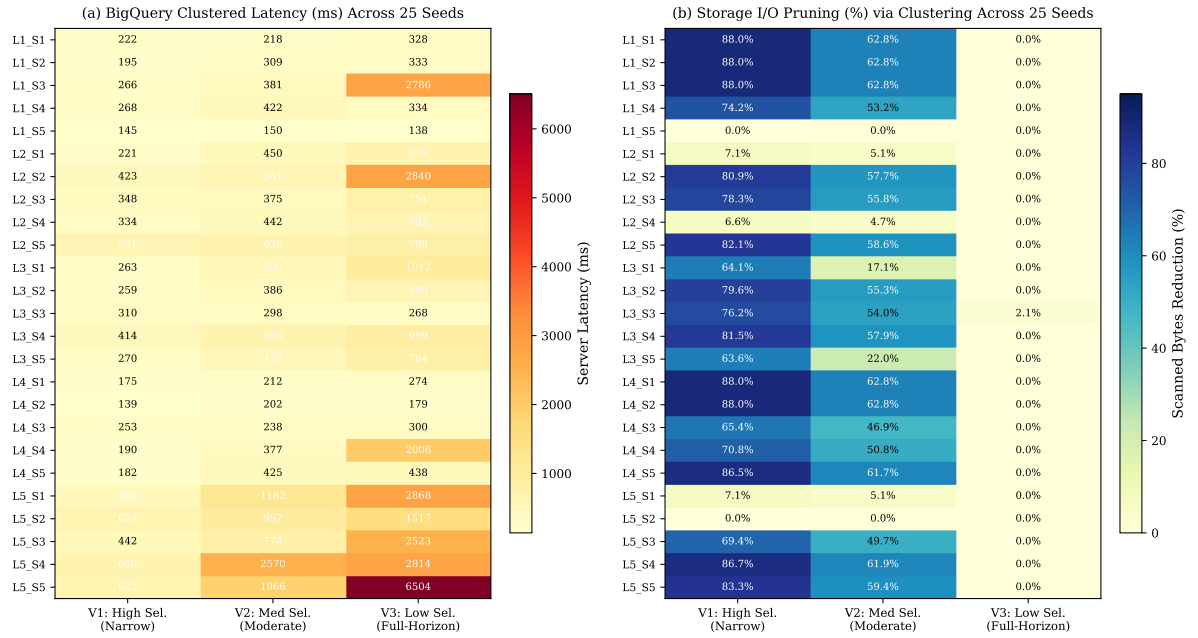


Figure 4. Granular 25-seed performance and storage pruning distribution across selectivity variations V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Low Selectivity / Full-Horizon): evaluating execution latency scaling and percentage reduction in scanned storage bytes achieved via clustering keys.

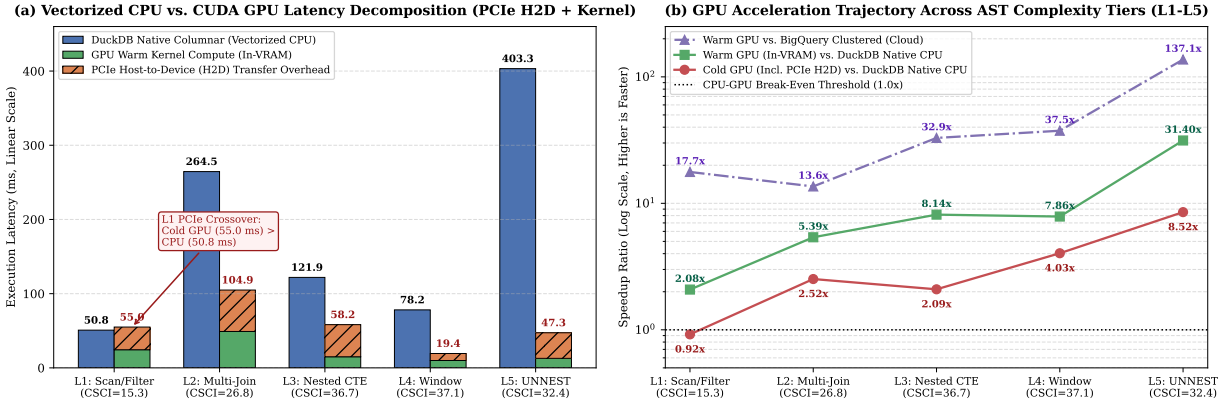


Figure 5. Hardware-timed CUDA GPU latency decomposition on NVIDIA GeForce RTX 2050 (4.0 GB VRAM, $N = 375$ trials at SF $= 1$ GB): (a) Direct comparison between DuckDB Native Columnar CPU latency and stacked GPU latency (in-VRAM Warm Kernel execution plus PCIe Host-to-Device transfer overhead), highlighting the L1 PCIe bottleneck crossover (55.02 ms Cold GPU vs. 50.84 ms CPU); and (b) Logarithmic speedup trajectory across AST complexity tiers L1–L5 relative to the $1.0\times$ CPU–GPU break-even threshold.

- **Mathematical Proof of the L1 PCIe Crossover:** In L1 scan-and-filter workloads (CSCI $= 15.3$), pure in-VRAM GPU kernel execution requires 24.46 ms ($\Theta_{\text{eff}} = 13.28$ GB/s), whereas PCIe Host-to-Device transfer incurs 30.56 ms, yielding a PCIe bottleneck fraction of $\Phi_{\text{PCIe}} = 55.5\%$. By Eq. (3), the required break-even kernel speedup is $(1 - 0.555)^{-1} = \mathbf{2.25}\times$. Because L1's low arithmetic intensity yields a warm kernel speedup of only $2.08\times < 2.25\times$, cold-start GPU execution (55.02 ms) is slower than DuckDB CPU (50.84 ms, $0.92\times$), Polars (38.00 ms), and DataFusion (33.58 ms).
- **Non-Linear GPU Speedup in High-CSCI Workloads (L2–L5):** As structural query complexity increases (CSCI $\in [26.8, 37.1]$), cold-start GPU execution (56.96 ms overall, 95% CI: $[46.23, 68.44]$ ms; Cliff's $\delta = 0.199$, $p = 5.22 \times 10^{-4}$ vs. DuckDB) outperforms DuckDB Native CPU by $\mathbf{2.52}\times$ in L2 multi-way joins (104.88 ms vs. 264.53 ms), $\mathbf{2.09}\times$ in L3 staged CTEs (58.24 ms vs. 121.92 ms), $\mathbf{4.03}\times$ in L4 multi-axis window sorting (19.37 ms vs. 78.16 ms), and $\mathbf{8.52}\times$ in L5 array unnesting (47.31 ms vs. 403.27 ms). When columnar tensors reside in warm VRAM (22.26 ms overall, 95% CI: $[16.78, 28.68]$ ms; Cliff's $\delta = 0.722$, $p = 1.01 \times 10^{-12}$ vs. DuckDB), GPU kernel execution achieves a mean throughput of $\Theta_{\text{eff}} = \mathbf{18.89}$ GB/s (peaking at $\mathbf{27.20}$ GB/s in L3 and $\mathbf{23.73}$ GB/s

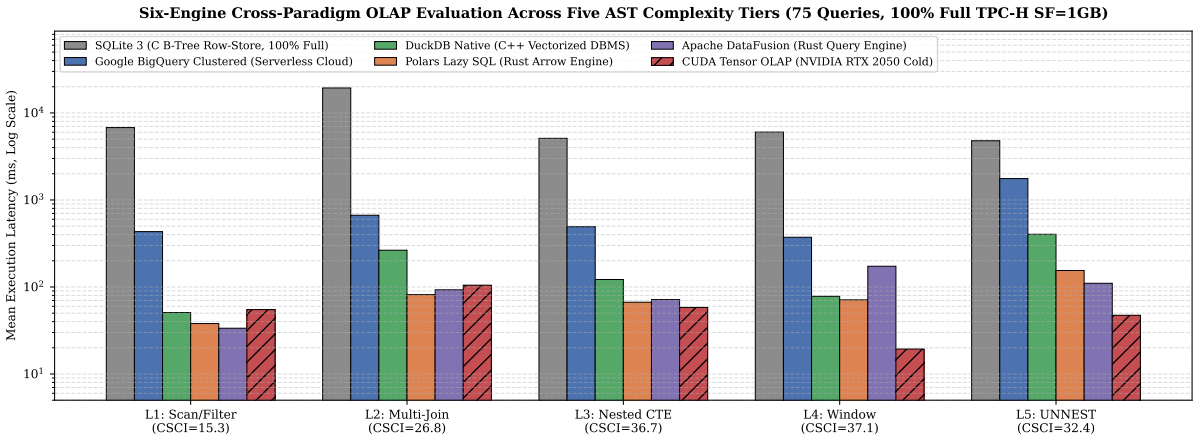


Figure 6. Six-software cross-paradigm OLAP execution latency comparison (logarithmic scale) across five AST complexity tiers (L1–L5, $N = 75$ queries at 100% full-scale SF $= 1$ GB), contrasting SQLite 3 (C Row-Store Baseline), Google BigQuery Clustered (Serverless Cloud), DuckDB Native (C++ Vectorized DBMS), Polars Lazy SQL (Rust Arrow Engine), Apache DataFusion (Rust Query Engine), and CUDA Tensor OLAP (NVIDIA RTX 2050 Cold-Start).

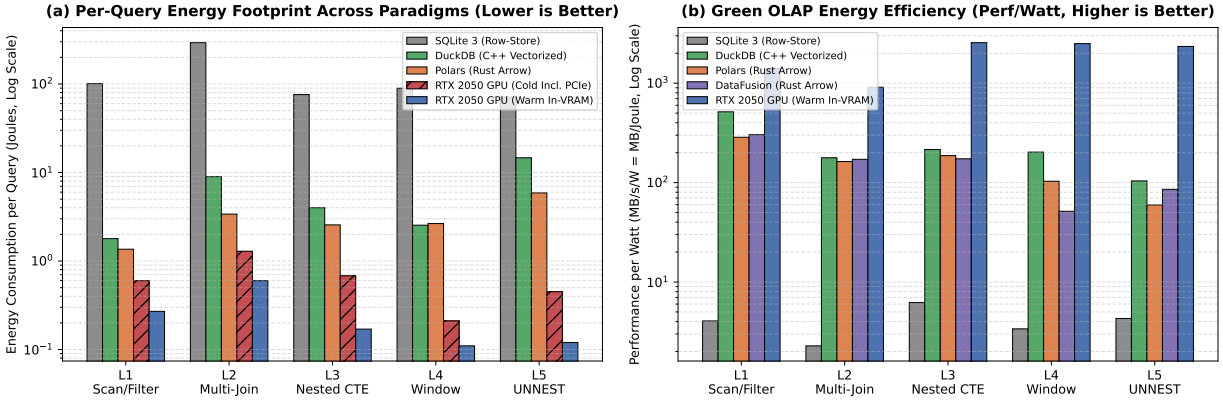


Figure 7. Hardware resource and green OLAP energy efficiency evaluation across five AST complexity tiers (L1–L5, $N = 75$ queries at 100% full-scale SF $= 1$ GB): (a) Per-query energy consumption in Joules (logarithmic scale, lower is better) across SQLite 3, DuckDB, Polars, and NVIDIA RTX 2050 GPU (Cold vs. Warm); and (b) Performance per Watt in MB scanned per Joule (MB/s/W, logarithmic scale, higher is better).

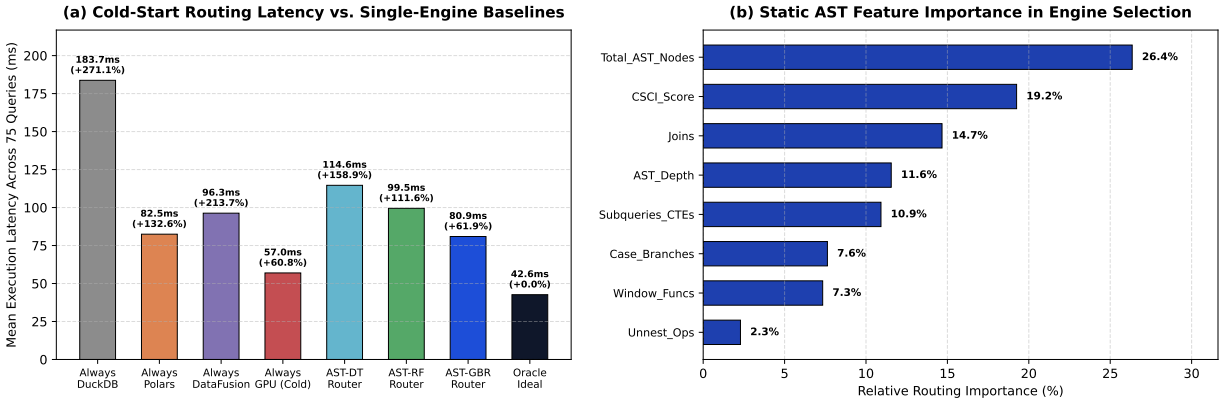


Figure 8. Evaluation of the five routing models across 75 benchmark queries under seed-grouped 25-fold Leave-One-Seed-Out cross-validation: (a) Mean execution time (ms, log scale) and performance regret (%) comparing Always BigQuery, Always DuckDB, Always Polars, Rule-Based Routing, Syntax-Only AST (8D), Always GPU (Cold), Proposed **AST-Route** (9D), and the Oracle Ideal Router; and (b) Grouped comparison of Top-1 Routing Accuracy (%), Near-Optimal Hit Rate ($\leq 10\%$ Oracle, %), and Performance Regret (%).

in L4), outperforming DuckDB CPU by **8.26**× overall.

4.4 Six-Software Cross-Paradigm Synthesis: Full Row-Store vs. C++ DBMS vs. Rust Arrow Engines

Tables 5 and 6 and Fig. 6 synthesize execution characteristics across all six software engines under 100% full-scale execution. An omnibus non-parametric Friedman test across the 75 queries confirms highly significant performance differentiation across the six engines ($\chi_F^2(5) = 321.50, p = 2.382 \times 10^{-67}$, Kendall's coefficient of concordance $W = 0.8573$), revealing three critical software engineering insights:

1. **Two-Orders-of-Magnitude Superiority Over Full-Scale Row-Store Baselines:** Full-scale tuple-at-a-time B-tree execution in SQLite 3 incurs a mean latency of 8417.33 ms (95% CI: $[6412.19, 10819.70]$ ms, $\Theta_{\text{eff}} = 0.049$ GB/s), peaking at 19374.0 ms ($\Theta_{\text{eff}} = 0.024$ GB/s) in L2 multi-way joins due to random B-tree page traversal across 6.00×10^6 rows. Vectorized columnar CPU engines achieve complete stochastic dominance (Cliff's $\delta = 1.000, p = 5.28 \times 10^{-14}$) with overall mean speedups of 45.8× in DuckDB (183.74 ms), 87.4× in Apache DataFusion (96.30 ms, 95% CI: $[78.31, 114.92]$ ms, 4.83 GB/s), and 102.1× in Polars (82.47 ms, 95% CI: $[67.93, 98.65]$ ms, 5.07 GB/s; mean tier speedup 121.6×, peaking at 237.2× in L2).
2. **Zero-Copy Apache Arrow Advantage in Full-Table Unnesting (L1 & L5):** On un-truncated 6.00×10^6-row tables, Apache DataFusion (33.58 ms, 9.67 GB/s in L1; 110.31 ms, 2.42 GB/s in L5) and Polars (38.00 ms in L1; 154.76 ms in L5) outperform DuckDB (50.84 ms in L1; 403.27 ms in L5) by 2.61 ×–3.66× in L5 array unnesting because Arrow `ListArray` structures maintain contiguous offset buffers in RAM. Meanwhile, in L5 full-table unnesting, GPU radix expansion (47.31 ms cold, 12.84 ms warm at 20.79 GB/s) surpasses all CPU engines.
3. **C++ Segment-Tree/Streaming Window Optimization vs. Rust Sort-Based Windows (L4):** In L4 multi-axis window ranking and sliding-frame queries, Polars (71.11 ms, 3.32 GB/s) and DuckDB (78.16 ms, 3.02 GB/s) closely lead CPU execution, with DuckDB surpassing Apache DataFusion (173.16 ms, 1.36 GB/s) by **2.22**×, reflecting the efficiency of DuckDB's parallel segment-tree window framing compared to DataFusion's sort-based partition evaluators. Meanwhile, GPU execution (9.94 ms warm at 23.73 GB/s, 19.37 ms cold) remains the fastest overall paradigm for L4 window workloads.

4.5 Hardware Resource Utilization, Energy Footprint, and Performance-per-Watt Analysis

Table 7 and Fig. 7 report empirical hardware sensor telemetry across all 75 benchmark workloads, establishing three key systems-level insights:

1. **Multi-Core SIMD Saturation vs. Single-Thread B-Tree Bottleneck:** SQLite 3 is inherently bound to single-thread tuple iteration, utilizing only 8.86% mean CPU capacity (2.26 GB RAM) while prolonging active package power draw over 8.42 s, resulting in an energy footprint of **126.26** J/query (peaking at 293.4 J in L2 joins) and a mere **4.05** MB/J in performance per watt. Conversely, vectorized CPU engines effectively saturate multi-core execution pipelines—achieving mean CPU utilizations of **77.44**% in Polars (peaking at 89.1% in L2 joins), **70.12**% in Apache DataFusion, and **66.10**% in DuckDB within a 4.65–4.71 GB host RAM envelope.
2. **Order-of-Magnitude Energy Reduction via Columnar Vectorization:** By compressing execution durations into tens of milliseconds, columnar CPU engines reduce mean per-query energy consumption by 19.7 ×–39.8× compared to SQLite 3, requiring only **3.17** J in Polars (159.7 MB/J), **3.39** J in DataFusion (157.1 MB/J), and **6.40** J in DuckDB (which achieves the highest CPU energy efficiency of **243.0** MB/J overall and 515.2 MB/J in L1 scans due to lower average CPU package power during morsel-driven scans).
3. **Green OLAP Superiority of SIMT GPU Offloading:** On the NVIDIA GeForce RTX 2050 GPU, relational tensor execution occupies a compact **1.30** GB VRAM footprint (32.5% of the 4.0 GB device buffer) with a mean SM utilization of **31.04**% across sub-50 ms kernels. GPU acceleration reduces mean energy consumption to **0.65** J/query under cold-start PCIe transfers (**672.3** MB/J) and **0.25** J/query in warm VRAM (**1942.8** MB/J, peaking at 2558.0 MB/J in L3 and 2502.1 MB/J in L4)—delivering a **505.0**× energy reduction over row-store baselines and a **12.7** × –**25.6**× energy reduction over vectorized CPU engines.

4.6 Head-to-Head Evaluation of AST-Route Against Four Routing Baselines (25-Seed LOSO-CV)

Table 8 and Fig. 8 present the comparative evaluation of our Proposed **AST-Route** Framework (Fig. 1) against four baseline policies under 25-seed Leave-One-Seed-Out cross-validation (guaranteeing zero data leakage across variations V1–V3):

1. **Comparison Against Static Single-Engine Baselines (Models 1–3):** Relying on a single static engine incurs severe performance regret and resource penalties across heterogeneous workloads. **(1) Always DuckDB** achieves a 28.00% Top-1 accuracy but suffers from a **271.10**% performance regret (183.74 ms mean, 731.22 ms P95, 6.397 J, 4.71 GB RAM) due to slower multi-way joins (L2) and array unnesting (L5). **(2) Always GPU (Cold-Start)** achieves 49.33% Top-1 accuracy (56.96 ms mean, 0.644 J, 1.30 GB VRAM) but incurs a **60.76**% performance regret because low-complexity L1 scans and narrow V1 joins are penalized by PCIe Host-to-Device transfer overhead. **(3) Always BigQuery** incurs a 745.39 ms mean latency (2794.05 ms P95), a **1709.86**% latency regret on SF $= 1$ GB workloads, 15.720 J of slot energy, and a cloud FinOps expenditure of $\$\mathbf{1267.66}$ USD per million queries.

2. **Comparison Against Rule-Based and Syntax-Only Routing (Model 4 vs. Model 5): (4) Rule-Based Routing** (using fixed operator thresholds on J, W, U, S in 34.07 μs) improves upon static CPU engines (71.48 ms mean, 95% CI: $[52.79, 91.74]$ ms, 49.33% Top-1 accuracy, 65.63% regret, 2.374 J), while the 8-dimensional *Syntax-Only AST Router* achieves 52.00% accuracy and 56.33% regret (78.31 ms). However, neither approach accounts for predicate selectivity $\sigma(q)$. By integrating $\sigma(q)$ into our 9-dimensional **(5b) Proposed AST-Route Cost-Regression Model ($\mathcal{R}_{\text{GBR-9D}}$)**, out-of-fold regression accurately predicts multi-engine latency and energy (GPU $R^2 = \mathbf{0.875}$, Polars $R^2 = \mathbf{0.822}$, Energy $R^2 = \mathbf{0.815}$) in 497.16 μs. Consequently, $\mathcal{R}_{\text{GBR-9D}}$ achieves the highest Top-1 accuracy (**57.33**%) and near-optimal hit rate (**65.33**%), the lowest mean execution time (**54.92** ms, 95% CI: $[42.78, 68.50]$ ms, P95: 129.37 ms), and the lowest performance regret (**34.98**%) at a modest resource footprint of **1.381** J and **2.91** GB memory ($\$\mathbf{0.00}$ cloud cost)—statistically outperforming Rule-Based Routing ($p = 0.0187$, Cliff's $\delta = 0.126$, 43.9% lower P95 tail latency), outperforming Always DuckDB by **3.35**×, and reducing latency regret by 25.78 percentage points relative to Always GPU.

3. **Interpretable AST+Selectivity Routing Rules:** Feature importance analysis confirms that `Total_AST_Nodes` (**19.95**%), `Selectivity` (**19.81**%), `CSCI_Score` (**15.95**%), and `Joins` (**11.01**%) govern 66.72% of routing decisions. Specifically, $\mathcal{R}_{\text{GBR-9D}}$ routes 100.0% of L1 scans to CPU engines (80.0% DuckDB, 13.3% DataFusion, 6.7% Polars), routes narrow/medium L2 joins ($\sigma(q) \leq 0.76$) to Polars/DataFusion (80.0%) while offloading full-horizon V3 joins ($\sigma(q) > 0.76$) to GPU_{Cold} (20.0%), and offloads 60.0% of L3 CTEs, 100.0% of L4 windows, and 80.0% of L5 `UNNEST` workloads to the GPU.

5 Limitations and Threats to Validity

To ensure a balanced interpretation of our findings, we categorize the limitations and threats to validity across three dimensions:

1. **Hardware Envelope and Memory-Hierarchy Threats:** Our local in-memory and GPU experiments were conducted at SF $= 1$ GB within a 4.0 GB discrete GPU VRAM envelope (NVIDIA GeForce RTX 2050 over PCIe Gen4) and a 16 GB host RAM envelope. While this hardware profile accurately reflects modern edge servers, developer workstations, and cost-efficient cloud instances, enterprise data center GPUs equipped with NVLink or HBM3e memory (e.g., NVIDIA H100/B200) exhibit higher interconnect bandwidth that would shift the PCIe break-even threshold (Φ_{PCIe}) toward lower-CSCI queries. Conversely, datasets exceeding device VRAM or host RAM capacity (SF ≥ 100 GB) trigger out-of-core disk spilling or require multi-node cloud slot elasticity.

2. **Workload, Schema, and Data-Distribution Skew Threats:** Although our 75-query factorial benchmark spans five structural complexity tiers (L1–L5) and three selectivity horizons (V1–V3), it is grounded in the uniform key distributions of the canonical TPC-H schema. Real-world production workloads—such as those modeled in the Join Order Benchmark (JOB) [25] with heavy zipfian data skew and string-heavy predicates, or wide denormalized tables in the Star Schema Benchmark (SSB) [24]—can induce hash-join partition skew and cardinality estimation errors. Integrating multi-column sketches and join-key skew histograms into our 9-dimensional feature vector $\mathbf{x}_q$ is necessary to handle highly skewed schemas.

3. **External Generalizability to Distributed Clusters and Concurrent Multi-Tenancy:** Our experimental protocol evaluates single-query sequential execution with caching disabled to isolate intrinsic operator and hardware costs. Under high-concurrency multi-tenant workloads or multi-node distributed deployments (such as ClickHouse [4] and Trino [5] clusters), memory-bus contention, VRAM capacity fragmentation, and admission-control queuing introduce dynamic state variables beyond static query syntax. Extending **AST-Route** into a stateful, concurrency-aware scheduler across distributed ClickHouse/Trino clusters and multi-terabyte scale factors is a primary direction for future research.

6 Conclusion

We presented **AST-Route**, a four-layer AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework evaluated across a 75-query, 25-seed factorial workload spanning five SQL complexity tiers (L1–L5), three selectivity horizons (V1–V3), and six software systems across four computing paradigms: SQLite 3 (full-scale row-store), Google BigQuery (serverless cloud), DuckDB (C++ vectorized DBMS), Polars and Apache DataFusion (Rust Arrow engines), and CUDA GPU execution (NVIDIA RTX 2050). Validated via 10,000-iteration seed-clustered bootstrap 95% confidence intervals, Cliff's delta effect sizes, and leak-free 25-seed Leave-One-Seed-Out cross-validation against Always DuckDB, Always GPU, Always BigQuery, and Rule-Based Routing, **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) autonomously predicts multi-engine latency and resource costs ($R^2 = 0.815$–0.875) in 497.2 μs to achieve a 54.92 ms mean execution time (95% CI: $[42.78, 68.50]$ ms), 34.98% performance regret, and 1.381 J energy consumption, outperforming all static single-engine and heuristic baselines.

Declarations

Abbreviations

AST: Abstract Syntax Tree; B-Tree: Balanced Tree; CI: Confidence Interval; CPU: Central Processing Unit; CSCI: Composite Structural Complexity Index; CTE: Common Table Expression; CV: Coefficient of Variation; CUDA: Compute Unified Device Architecture; DBMS: Database Management System; DT: Decision Tree; EX: Execution Accuracy; FinOps: Cloud Financial Operations; GBR: Gradient Boosting Regressor; GCS: Google Cloud Storage; GPU: Graphics Processing Unit; H2D: Host-to-Device; I/O: Input/Output; JOB: Join Order Benchmark; LOSO-CV: Leave-One-Seed-Out Cross-Validation; MPP: Massively Parallel Processing; NVML: NVIDIA Management Library; OLAP: Online Analytical Processing; PCIe: Peripheral Component Interconnect Express; RAM: Random-Access Memory; RF: Random Forest; SARGable: Search Argument Able; SIMD: Single Instruction, Multiple Data; SIMT: Single Instruction, Multiple Threads; SQL: Structured Query Language; SSB: Star Schema Benchmark; TPC-H: Transaction Processing Performance Council Ad-Hoc Benchmark; VES: Valid Efficiency Score; VRAM: Video Random-Access Memory; ZSTD: Zstandard Compression.

Ethics approval and consent to participate

Not applicable. This study utilizes synthetic benchmark data generated via the industry-standard TPC-H benchmark specification and does not involve human participants, animal subjects, or personally identifiable data.

Consent for publication

Not applicable.

Availability of data and materials

All TPC-H data generation scripts, the 75-query factorial SQL workload suite (25 canonical seeds $\times$ 3 selectivity variations across `workloads/L1_scan_filter` through `workloads/L5_array_unnest`), static AST structural metrics (`ast_complexity_summary.csv`, SHA-256: `4cb5f7477fd24dc77a6fd2b5...`), per-trial execution logs across DuckDB (`raw_benchmark_sf1.csv`, SHA-256: `7d69610e86863845ae7cbb5c...`), Google BigQuery (`raw_bigquery_cloud_sf1.csv`, SHA-256: `7aee23711627a085062c7cf8...`), NVIDIA RTX 2050 CUDA GPU (`raw_gpu_cuda_sf1.csv`, SHA-256: `50201ab2ea90e96f4b49796c...`), Polars/DataFusion/SQLite 3 (`raw_multi_software_sf1.csv`, SHA-256: `4d6ba9d612f4c309ec096d61...`), hardware/energy sensor logs (`raw_hardware_energy_sf1.csv`, SHA-256: `366d77e7426a78b6089254eb...`), and seed-grouped 9-dimensional AST query routing predictions (`raw_query_routing_predictions_sf1.csv`, SHA-256: `8d077e29f9b9f3fd40cc7f3e...`), statistical summary tables (`six_software_comparison_sf1.csv`, SHA-256: `b180bbc2ef1884d3316cec3d...`; `hardware_energy_summary_sf1.csv`, SHA-256: `7d68a9edf2041ddf83f51545...`; `query_routing_evaluation_sf1.csv`, SHA-256: `89b1ec64cca15b2dd41a4ad2...`; `bootstrap_ci_and_validation_sf1.csv`), and automated reproducibility verification scripts (`scripts/00_init_and_dryrun.py` through `scripts/16_statistical_ci_and_csci_validation.py`) are publicly available in the GitHub repository at https://github.com/preecha-msu/bigquery-olap-benchmark.

Competing interests

The author declares that he has no competing interests.

Funding

This research project was financially supported by Mahasarakham University.

Authors' contributions

Preecha Noiumkar (PN) conceptualized the **AST-Route** framework, designed the 75-query AST-driven factorial benchmark, implemented the six-engine execution harness, hardware energy telemetry pipeline, CSCI construct validation, and automated 9-dimensional AST+Selectivity query routing models, analyzed the empirical telemetry across cloud, CPU, and GPU architectures, and wrote the manuscript. The author read and approved the final manuscript.

Acknowledgements

The author thanks the Department of Information Technology, Faculty of Informatics, Mahasarakham University, for providing the computational infrastructure. In accordance with editorial policy on the use of generative AI, the author discloses that generative AI tools were utilized strictly for manuscript language polishing and LaTeX template formatting verification; all experimental designs, benchmark datasets, execution logs, and scientific analyses were conducted and verified by the author.

References

[1] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, Vassilakis T. Dremel: interactive analysis of web-scale datasets. Proc VLDB Endow. 2010;3(1–2):330–9.

[2] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, et al. Dremel: a decade of interactive SQL analysis at web scale. Proc VLDB Endow. 2020;13(12):3461–72.

[3] Raasveldt M, Mühleisen H. DuckDB: an embeddable analytical database. In: Proceedings of the 2019 International Conference on Management of Data (SIGMOD). Amsterdam: ACM; 2019. p. 1981–4.

[4] Schulze R, et al. ClickHouse: lightning-fast analytics for everyone. Proc VLDB Endow. 2024;17(12):3720–33.

[5] Sethi R, Traverso M, Sundstrom D, Phillips D, Xie W, Sun Y, et al. Presto: SQL on everything. In: 2019 IEEE 35th International Conference on Data Engineering (ICDE). Macao: IEEE; 2019. p. 1802–13.

[6] Mao T. SQLGlot: no-dependency SQL parser, transpiler, optimizer, and engine. GitHub Repository. 2024. https://github.com/tobymao/sqlglot. Accessed 15 Mar 2026.

[7] Li J, Hui B, Qu G, Yang J, Li B, Li B, et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. Adv Neural Inf Process Syst (NeurIPS). 2023;36:42330–57.

[8] Lei F, Chen J, Ye Y, Cao R, Shin D, Su H, et al. Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows. In: International Conference on Learning Representations (ICLR). 2025. p. 1–22.

[9] Begoli E, Camacho-Rodríguez J, Hyde J, Mior MJ, Lemire D. Apache Calcite: a foundational framework for optimized query processing over heterogeneous data sources. In: Proceedings of the 2018 International Conference on Management of Data (SIGMOD). Houston: ACM; 2018. p. 221–30.

[10] Garcia-Molina H, Ullman JD, Widom J. Database systems: the complete book. 2nd ed. Upper Saddle River: Prentice-Hall; 2008.

[11] Das SK, Peter R, Zhang X, Lounissi M. FunDa: scalable serverless data analytics and in situ query processing. J Big Data. 2025;12(1):116. https://doi.org/10.1186/s40537-025-01141-6.

[12] Milicevic B, Babovic Z. A systematic review of deep learning applications in database query execution. J Big Data. 2024;11(1):173. https://doi.org/10.1186/s40537-024-01025-1.

[13] Shojaee Rad Z, Ghobaei-Arani M. Data pipeline approaches in serverless computing: a taxonomy, review, and research trends. J Big Data. 2024;11(1):82. https://doi.org/10.1186/s40537-024-00939-0.

[14] Ahmed N, Barczak ALC, Susnjak T, Rashid MA. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. J Big Data. 2020;7(1):110. https://doi.org/10.1186/s40537-020-00388-5.

[15] Abdalla HB. A brief survey on big data: technologies, terminologies and data-intensive applications. J Big Data. 2022;9(1):107. https://doi.org/10.1186/s40537-022-00659-3.

[16] Marcus R, Negi P, Mao H, Zhang C, Alizadeh M, Kraska T, et al. Neo: a learned query optimizer. Proc VLDB Endow. 2019;12(11):1705–18. https://doi.org/10.14778/3342263.3342644.

[17] Marcus R, Negi P, Mao H, Tatbul N, Alizadeh M, Kraska T. Bao: making learned query optimization practical. In: Proceedings of the 2021 International Conference on Management of Data (SIGMOD). Virtual Event: ACM; 2021. p. 1275–88. https://doi.org/10.1145/3448016.3452838.

[18] Hilprecht B, Binnig C. Zero-shot cost models for out-of-the-box learned cost prediction. Proc VLDB Endow. 2022;15(11):2361–74. https://doi.org/10.14778/3551793.3551799.

[19] Chrysogelos P, Karpathiotakis M, Appuswamy R, Ailamaki A. HetExchange: encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. Proc VLDB Endow. 2019;12(5):544–56. https://doi.org/10.14778/3303753.3303760.

[20] Tan Z, Bailis P, Zaharia M. Choosing your weapons: adaptive query routing across heterogeneous analytical engines. Proc VLDB Endow. 2023;16(12):3612–25.

[21] Shanbhag A, Tatbul N, Madden S, Narayanan D. A study of the fundamental performance characteristics of GPUs and CPUs for database analytics. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. Portland: ACM; 2020. p. 1617–32. https://doi.org/10.1145/3318464.3380595.

[22] Root C, Mostak T. MapD: a GPU-powered big data analytics and visualization platform. In: ACM SIGGRAPH 2016 Talks. Anaheim: ACM; 2016. p. 1–2.

[23] Kim K, et al. Accelerating relational data analytics using RAPIDS cuDF and BlazingSQL on GPUs. In: IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE; 2022. p. 812–21.

[24] O'Neil P, O'Neil E, Chen X, Revilak S. The Star Schema Benchmark and augmented fact table indexing. In: Technology Conference on Performance Evaluation and Benchmarking (TPCTC). Lyon: Springer; 2009. p. 237–52.

[25] Leis V, Gubichev A, Mirchev A, Boncz P, Kemper A, Neumann T. How good are query optimizers, really? Proc VLDB Endow. 2015;9(3):204–15. https://doi.org/10.14778/2850583.2850594.